Python OOP Basics

You said:

complete python oops with code example and very easy def and tough word meaning with very easy

ChatGPT said:

Complete Python OOP — Easy Definitions and Code

1. What is OOP?

OOP means **Object-Oriented Programming**.

It is a programming style in which we organize code using:

- Classes
- Objects
- Data
- Methods

Easy real-life example

A **class** is like a blueprint of a house.
An **object** is the actual house created from that blueprint.

```
class Car:
    pass

car1 = Car()
car2 = Car()

print(car1)
print(car2)
```

car1 and car2 are two different objects of the Car class.

2. Important Difficult Words

Difficult word	Very easy meaning
Blueprint	A plan used to create something
Class	A design used to create objects

Difficult word	Very easy meaning
Object	A real item created from a class
Instance	Another name for an object
Attribute	A variable belonging to an object
Method	A function belonging to a class
Behaviour	What an object can do
State	Current data stored in an object
Initialize	Give starting values
Instantiate	Create an object
Encapsulation	Protect and group data
Inheritance	Get features from another class
Polymorphism	Same action with different behaviour
Abstraction	Hide unnecessary internal details
Override	Rewrite a parent method
Access modifier	Controls how data is accessed
Constructor	Special method that initializes an object
Dunder method	Special method with double underscores
Interface	Rules describing required methods
MRO	Order in which Python searches parent classes

3. Class and Object

Class

A class is a blueprint that contains variables and methods.

Object

An object is a real instance created from a class.

```
class Student:
    name = "Aamir"

    def study(self):
        print("Student is studying")

student1 = Student()

print(student1.name)
student1.study()
```

Output:

```
Aamir
Student is studying
```

Here:

- Student is a class.
- student1 is an object.
- name is an attribute.
- study() is a method.

---

4. Constructor: __init__()

A constructor is a special method that runs automatically when an object is created.

In Python, __init__() initializes the object.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Aamir", 24)
student2 = Student("Rahul", 22)

print(student1.name, student1.age)
print(student2.name, student2.age)
```

Output:

```
Aamir 24
Rahul 22
```

Dry run

```
student1 = Student("Aamir", 24)
```

Python performs:

1. Creates a new `Student` object.
2. Calls `__init__()`.
3. `self` points to `student1`.
4. `"Aamir"` is stored in `self.name`.
5. `24` is stored in `self.age`.

> Technically, `__new__()` creates the object and `__init__()` initializes it. In interviews, `__init__()` is commonly called the constructor.

5. What is `self`?

`self` represents the **current object.**

```
class Employee:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Employee:", self.name)

employee1 = Employee("Aamir")
employee2 = Employee("Ali")

employee1.show()
employee2.show()
```

Output:

```
Employee: Aamir
Employee: Ali
```

Internally:

```
employee1.show()
```

is similar to:

```
Employee.show(employee1)
```

self is not a keyword, but using the name self is the standard convention.

---

6. Attributes in Python

An attribute is a variable belonging to a class or object.

6.1 Instance variable

An instance variable belongs to one particular object.

```
class Student:
    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.name)
print(student2.name)
```

Every object can have a different value.

6.2 Class variable

A class variable is shared by all objects.

```
class Student:
    college = "UEM Jaipur"

    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.college)
print(student2.college)
print(Student.college)
```

Difference

Instance variable	Class variable
Belongs to an object	Belongs to the class
Created using self	Written directly inside class
Usually different for every object	Shared by all objects
Example: student name	Example: college name

7. Types of Methods

Python classes mainly have three types of methods:

1. Instance method
2. Class method
3. Static method

7.1 Instance method

Works with object-specific data and uses self.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print(self.name, self.marks)

student = Student("Aamir", 85)
student.display()
```

7.2 Class method

Works with class-level data and uses cls.

It uses the @classmethod decorator.

```
class Student:
    college = "UEM Jaipur"

    @classmethod
    def change_college(cls, new_college):
        cls.college = new_college

Student.change_college("IIT Delhi")

print(Student.college)
```

Output:

```
IIT Delhi
```

7.3 Static method

A static method is a utility function related to the class.

It does not require self or cls.

```
class Calculator:
    @staticmethod
    def add(a, b):
        return a + b

print(Calculator.add(10, 20))
```

Output:

```
30
```

Comparison

Method	First parameter	Works with
Instance method	self	Object data
Class method	cls	Class data
Static method	Nothing compulsory	Utility logic

8. Four Pillars of OOP

The four main principles are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

9. Encapsulation

Easy definition

Encapsulation means:

- Keeping data and methods together inside a class.
- Controlling how data is accessed or modified.

Example: An ATM does not allow you to change your bank balance directly.

Public member

Public data can be accessed from anywhere.

```
class Employee:
    def __init__(self, name):
        self.name = name

employee = Employee("Aamir")

print(employee.name)
employee.name = "Rahul"
print(employee.name)
```

Protected member

A protected member starts with one underscore _.

```
class Employee:
    def __init__(self):
        self._salary = 50000

employee = Employee()
print(employee._salary)
```

It can still be accessed, but _salary tells programmers:

> This is for internal use. Do not access it directly unless necessary.

Private member

A private member starts with two underscores __.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def show_balance(self):
        print("Balance:", self.__balance)

account = BankAccount(10000)
account.show_balance()
```

This gives an error:

```
print(account.__balance)
```

Python changes its internal name using **name mangling.**

```
print(account._BankAccount__balance)
```

However, directly accessing it this way is not recommended.

> Python does not provide completely strict private variables. Double underscores mainly prevent accidental access.

Getters and setters

A getter reads private data.
A setter updates private data safely.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

    def set_balance(self, amount):
        if amount >= 0:
            self.__balance = amount
        else:
            print("Balance cannot be negative")

account = BankAccount(10000)

print(account.get_balance())

account.set_balance(15000)
print(account.get_balance())

account.set_balance(-500)
```

Using @property

@property provides a cleaner getter and setter.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, amount):
        if amount < 0:
            raise ValueError("Salary cannot be negative")

        self.__salary = amount

employee = Employee(50000)

print(employee.salary)

employee.salary = 60000
print(employee.salary)
```

It looks like normal variable access, but internally methods are running.

10. Inheritance

Easy definition

Inheritance means creating a new class from an existing class.

The child class receives properties and methods from the parent class.

```
class Person:
    def speak(self):
        print("Person can speak")

class Student(Person):
    def study(self):
        print("Student can study")

student = Student()

student.speak()
student.study()
```

Here:

- Person is the parent class.
- Student is the child class.
- Student inherits speak().

Benefits

- Code reuse
- Less duplicate code
- Easy maintenance
- Better organization

---

11. Types of Inheritance

11.1 Single inheritance

One child inherits from one parent.

```
class Parent:
    def house(self):
        print("Parent's house")

class Child(Parent):
    def bike(self):
        print("Child's bike")

child = Child()
child.house()
child.bike()
```

11.2 Multilevel inheritance

A class inherits through multiple levels.

```
class Grandparent:
    def property(self):
        print("Grandparent's property")

class Parent(Grandparent):
    def car(self):
        print("Parent's car")

class Child(Parent):
    def bike(self):
        print("Child's bike")

child = Child()

child.property()
child.car()
child.bike()
```

Relationship:

```
Grandparent → Parent → Child
```

11.3 Multiple inheritance

One child inherits from multiple parents.

```
class Father:
    def gardening(self):
        print("Father knows gardening")

class Mother:
    def cooking(self):
        print("Mother knows cooking")

class Child(Father, Mother):
    def coding(self):
        print("Child knows coding")

child = Child()

child.gardening()
child.cooking()
child.coding()
```

11.4 Hierarchical inheritance

Multiple child classes inherit from one parent.

```
class Animal:
    def eat(self):
        print("Animal is eating")

class Dog(Animal):
    def bark(self):
        print("Dog is barking")

class Cat(Animal):
    def meow(self):
        print("Cat is meowing")

dog = Dog()
cat = Cat()

dog.eat()
dog.bark()

cat.eat()
cat.meow()
```

11.5 Hybrid inheritance

Hybrid inheritance combines two or more inheritance types.

```
class A:
    def method_a(self):
        print("Method A")

class B(A):
    def method_b(self):
        print("Method B")

class C(A):
    def method_c(self):
        print("Method C")

class D(B, C):
    def method_d(self):
        print("Method D")

obj = D()

obj.method_a()
obj.method_b()
obj.method_c()
obj.method_d()
```

12. The super() Function

super() calls a method from the parent class.

```
class Person:
    def __init__(self, name):
        self.name = name

    def display(self):
        print("Name:", self.name)

class Employee(Person):
    def __init__(self, name, salary):
        super().__init__(name)
        self.salary = salary

    def display(self):
        super().display()
        print("Salary:", self.salary)

employee = Employee("Aamir", 50000)
employee.display()
```

Output:

```
Name: Aamir
Salary: 50000
```

Without `super().__init__(name)`, the parent's initialization would not run automatically.

---

13. Method Overriding

Easy definition

Method overriding happens when a child class provides its own version of a parent method.

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

animal = Animal()
dog = Dog()

animal.sound()
dog.sound()
```

Output:

```
Animal makes a sound
Dog barks
```

The child's `sound()` replaces the inherited version for `Dog` objects.

---

14. Method Resolution Order — MRO

MRO means the order in which Python searches classes for a method.

```
class A:
    def show(self):
        print("Class A")

class B(A):
    def show(self):
        print("Class B")

class C(A):
    def show(self):
        print("Class C")

class D(B, C):
    pass

obj = D()
obj.show()

print(D.mro())
```

Output:

```
Class B
[<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>]
```

Python searches in this order:

```
D → B → C → A → object
```

---

15. Polymorphism

Easy definition

Polymorphism means:

> One method name can perform different actions for different objects.

poly means many and morph means forms.

15.1 Polymorphism through methods

```
class Dog:
    def sound(self):
        print("Dog barks")

class Cat:
    def sound(self):
        print("Cat meows")

class Cow:
    def sound(self):
        print("Cow moos")

animals = [Dog(), Cat(), Cow()]

for animal in animals:
    animal.sound()
```

Output:

```
Dog barks
Cat meows
Cow moos
```

15.2 Duck typing

Duck typing means Python focuses on what an object can do, not its exact class.

> If an object has the required method, Python can use it.

```
class Bird:
    def fly(self):
        print("Bird is flying")

class Aeroplane:
    def fly(self):
        print("Aeroplane is flying")

def start_flying(item):
    item.fly()

start_flying(Bird())
start_flying(Aeroplane())
```

start_flying() does not check whether the object is a bird or aeroplane. It only expects a fly() method.

---

16. Method Overloading

Easy definition

Method overloading means having methods with the same name but different parameters.

Python does not support traditional method overloading like Java.

This does not work as expected:

```
class Calculator:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c
```

The second method replaces the first method.

Solution using default arguments

```
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c

calculator = Calculator()

print(calculator.add(10, 20))
print(calculator.add(10, 20, 30))
```

Output:

```
30
60
```

Solution using *args

```
class Calculator:
    def add(self, *numbers):
        return sum(numbers)

calculator = Calculator()

print(calculator.add(10, 20))
print(calculator.add(10, 20, 30, 40))
```

17. Operator Overloading

Operator overloading changes how an operator works for custom objects.

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __str__(self):
        return str(self.value)

number1 = Number(10)
number2 = Number(20)

result = number1 + number2
print(result)
```

Output:

```
30
```

Python internally converts:

```
number1 + number2
```

into:

```
number1.__add__(number2)
```

18. Abstraction

Easy definition

Abstraction means hiding complicated internal details and showing only necessary features.

Example: You drive a car using steering, accelerator and brake. You do not need to know every internal engine operation.

Python provides abstraction using the `abc` module.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    def start(self):
        print("Car starts with a key")

class Bike(Vehicle):
    def start(self):
        print("Bike starts with a self-start button")

car = Car()
bike = Bike()

car.start()
bike.start()
```

You cannot normally create an object of an incomplete abstract class:

```
vehicle = Vehicle()
```

It raises an error because `start()` has no implementation.

Terms

- **Abstract class:** A class designed mainly to be inherited.

- **Abstract method:** A method the child class must implement.
- **Concrete class:** A complete class whose object can be created.

19. Encapsulation vs Abstraction

Encapsulation	Abstraction
Protects and groups data	Hides complicated implementation
Focuses on data security	Focuses on necessary features
Uses private variables and properties	Uses abstract classes and methods
Example: protecting balance	Example: showing only withdraw()

20. Association, Aggregation and Composition

These concepts describe relationships between objects.

20.1 Association

Two independent objects work together.

```
class Teacher:
    def teach(self, student):
        print("Teacher is teaching", student.name)

class Student:
    def __init__(self, name):
        self.name = name

teacher = Teacher()
student = Student("Aamir")

teacher.teach(student)
```

Both objects can exist independently.

20.2 Aggregation

Aggregation means a weak **has-a** relationship.

The child object can exist without the parent object.

```
class Employee:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, name, employees):
        self.name = name
        self.employees = employees

employee1 = Employee("Aamir")
employee2 = Employee("Rahul")

department = Department("IT", [employee1, employee2])

for employee in department.employees:
    print(employee.name)
```

Employees can exist even if the department object is deleted.

20.3 Composition

Composition means a strong **has-a** relationship.

The parent creates and strongly controls the child object.

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start_car(self):
        self.engine.start()
        print("Car started")

car = Car()
car.start_car()
```

Here, the Car creates its own Engine.

Inheritance vs Composition

Inheritance	Composition
Represents an **is-a** relationship	Represents a **has-a** relationship
Dog is an Animal	Car has an Engine
Child gets parent methods	One object contains another object
Creates strong class connection	Usually provides greater flexibility

21. Dunder or Magic Methods

Dunder means **double underscore.**

Examples:

```
__init__
__str__
__len__
__add__
```

These methods allow custom objects to work with Python operations.

__str__()

Controls how an object is printed.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def __str__(self):
        return f"Student(name={self.name}, marks={self.marks})"

student = Student("Aamir", 85)
print(student)
```

Output:

```
Student(name=Aamir, marks=85)
```

__len__()

```
class Team:
    def __init__(self, members):
        self.members = members

    def __len__(self):
        return len(self.members)

team = Team(["Aamir", "Rahul", "Ali"])

print(len(team))
```

Output:

```
3
```

Common magic methods

Method	Purpose
__init__()	Initializes an object
__new__()	Creates a new object
__str__()	User-friendly object text
__repr__()	Developer-friendly object text
__len__()	Handles len()
__add__()	Handles +
__sub__()	Handles -
__eq__()	Handles ==
__lt__()	Handles <
__getitem__()	Handles indexing
__call__()	Makes an object callable

22. __str__() vs __repr__()

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Student name is {self.name}"

    def __repr__(self):
        return f"Student('{self.name}')"

student = Student("Aamir")

print(str(student))
print(repr(student))
```

Output:

```
Student name is Aamir
Student('Aamir')
```

__str__()	__repr__()
For normal users	For developers and debugging
Easy-to-read output	Detailed representation
Called by str() and print()	Called by repr()

23. __new__() vs __init__()

```
class Student:
    def __new__(cls, *args, **kwargs):
        print("__new__ creates the object")
        return super().__new__(cls)

    def __init__(self, name):
        print("__init__ initializes the object")
        self.name = name

student = Student("Aamir")
```

Output:

```
__new__ creates the object
__init__ initializes the object
```

__new__()	__init__()
Creates the object	Initializes the object
Runs first	Runs after __new__()
Receives cls	Receives self
Returns an object	Normally returns nothing

24. Destructor: __del__()

__del__() may run before an object is removed from memory.

```
class Demo:
    def __init__(self):
        print("Object created")

    def __del__(self):
        print("Object destroyed")

obj = Demo()
del obj
```

Do not depend on __del__() for important cleanup because its execution time is not always predictable. Context managers are usually safer for files and database connections.

25. Class Factory Method

A class method can create an object from another format.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, data):
        name, age = data.split("-")
        return cls(name, int(age))

student = Student.from_string("Aamir-24")

print(student.name)
print(student.age)
```

This is called an **alternative constructor** or **factory method.**

---

26. Checking Object and Class Types

isinstance()

Checks whether an object belongs to a class.

```
class Animal:
    pass

class Dog(Animal):
    pass

dog = Dog()

print(isinstance(dog, Dog))
print(isinstance(dog, Animal))
```

Output:

```
True
True
```

issubclass()

Checks whether one class inherits from another.

```
print(issubclass(Dog, Animal))
```

Output:

```
True
```

---

27. Object Identity and Equality

is

Checks whether two variables point to the same object.

==

Checks whether two values are equal.

```
list1 = [10, 20]
list2 = [10, 20]
list3 = list1

print(list1 == list2)
print(list1 is list2)
print(list1 is list3)
```

Output:

```
True
False
True
```

For custom objects, == can be controlled with __eq__().

```
class Student:
    def __init__(self, roll_number):
        self.roll_number = roll_number

    def __eq__(self, other):
        return self.roll_number == other.roll_number

student1 = Student(101)
student2 = Student(101)

print(student1 == student2)
```

Output:

```
True
```

28. Shallow Copy and Deep Copy

Shallow copy

Creates a new outer object, but nested objects may still be shared.

Deep copy

Creates completely independent copies, including nested objects.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
deep = copy.deepcopy(original)

original[0][0] = 100

print("Original:", original)
print("Shallow:", shallow)
print("Deep:", deep)
```

Output:

```
Original: [[100, 2], [3, 4]]
Shallow: [[100, 2], [3, 4]]
Deep: [[1, 2], [3, 4]]
```

29. Dataclass

A dataclass automatically creates methods such as __init__(), __repr__() and __eq__().

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    age: int
    marks: float

student1 = Student("Aamir", 24, 85.5)
student2 = Student("Aamir", 24, 85.5)

print(student1)
print(student1 == student2)
```

Output:

```
Student(name='Aamir', age=24, marks=85.5)
True
```

Use a dataclass when the class mainly stores data.

30. Complete Bank Account Example

This example combines constructor, encapsulation, inheritance, abstraction and polymorphism.

```
from abc import ABC, abstractmethod


class BankAccount(ABC):
    bank_name = "ABC Bank"

    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit must be positive")
            return

        self.__balance += amount
        print(f"₹{amount} deposited successfully")

    def _deduct_balance(self, amount):
        self.__balance -= amount

    @abstractmethod
    def withdraw(self, amount):
        pass

    def display(self):
        print("Bank:", self.bank_name)
        print("Account holder:", self.account_holder)
        print("Balance:", self.balance)


class SavingsAccount(BankAccount):
    def withdraw(self, amount):
        if amount <= 0:
            print("Withdrawal must be positive")
        elif amount > self.balance:
            print("Insufficient balance")
        else:
            self._deduct_balance(amount)
            print(f"₹{amount} withdrawn successfully")


class CurrentAccount(BankAccount):
    def __init__(self, account_holder, balance=0, overdraft_limit=5000):
        super().__init__(account_holder, balance)
        self.overdraft_limit = overdraft_limit
```

```
    def withdraw(self, amount):
        allowed_amount = self.balance + self.overdraft_limit

        if amount <= 0:
            print("Withdrawal must be positive")
        elif amount > allowed_amount:
            print("Overdraft limit exceeded")
        else:
            self._deduct_balance(amount)
            print(f"₹{amount} withdrawn successfully")


accounts = [
    SavingsAccount("Aamir", 10000),
    CurrentAccount("Rahul", 3000)
]

for account in accounts:
    account.deposit(2000)
    account.withdraw(4000)
    account.display()
    print()
```

Concepts used:

- BankAccount → abstract parent class
- SavingsAccount and CurrentAccount → inheritance
- __balance → encapsulation
- withdraw() → abstraction
- Different withdraw() implementations → polymorphism
- super() → parent initialization
- @property → controlled data access

---

31. Important OOP Differences

Class vs object

Class	Object
Blueprint	Real instance
Logical design	Occupies memory
Created using class	Created by calling the class

Class	Object
Example: Car	Example: car1

Attribute vs method

Attribute	Method
Stores data	Performs an action
Example: name	Example: display()

Overloading vs overriding

Overloading	Overriding
Same method name with different parameters	Child rewrites parent method
Not traditionally supported in Python	Fully supported
Achieved using defaults or *args	Achieved through inheritance

Instance variable vs class variable

Instance variable	Class variable
Belongs to each object	Shared by all objects
Uses self.variable	Written inside the class
Can differ between objects	Usually the same for all

Abstract class vs normal class

Abstract class	Normal class
May contain abstract methods	Contains implemented methods
Mainly designed for inheritance	Objects can be created directly
Uses ABC	Does not require ABC

32. Most Asked Interview Questions

1. What is OOP?

OOP is a programming style that organizes programs using classes and objects. It improves code reuse, security and maintenance.

2. What is a class?

A class is a blueprint that contains attributes and methods.

3. What is an object?

An object is an instance created from a class.

4. What is `self`?

`self` refers to the current object and is used to access its attributes and methods.

5. What is `__init__()`?

It is a special method that automatically initializes an object after its creation.

6. What are the four pillars of OOP?

Encapsulation, inheritance, polymorphism and abstraction.

7. Does Python support multiple inheritance?

Yes. A class can inherit from multiple parent classes.

8. What is method overriding?

It happens when a child class provides its own implementation of a parent method.

9. Does Python support method overloading?

Not in the traditional Java-style form. It can be simulated using default parameters, `*args` or other techniques.

10. What is `super()`?

`super()` is used to call methods from a parent class.

11. What is MRO?

MRO is the order in which Python searches classes for methods and attributes.

12. What is duck typing?

If an object provides the required method, Python can use it without checking its exact class.

13. What is encapsulation?

Encapsulation means grouping data and methods and controlling access to internal data.

14. What is abstraction?

Abstraction hides unnecessary implementation details and shows only required features.

15. What is polymorphism?

Polymorphism allows the same method or operation to behave differently for different objects.

16. Is a private variable completely private in Python?

No. Double underscores use name mangling, but the value can still be accessed using its changed internal name.

17. What is the difference between `is` and `==`?

`is` checks object identity. `==` checks value equality.

18. What is composition?

Composition means one class strongly contains another object, such as a car containing an engine.

19. What is a static method?

A static method is a utility method that does not use object or class data directly.

20. What is a class method?

A class method works with class-level data and receives `cls` as its first parameter.

33. Final OOP Revision Map

```
OOP
├── Class and Object
├── Constructor and self
├── Attributes
│   ├── Instance variable
│   └── Class variable
├── Methods
│   ├── Instance method
│   ├── Class method
│   └── Static method
├── Encapsulation
│   ├── Public
│   ├── Protected
│   ├── Private
│   └── Property
├── Inheritance
│   ├── Single
│   ├── Multilevel
│   ├── Multiple
│   ├── Hierarchical
│   └── Hybrid
├── Polymorphism
│   ├── Overriding
│   ├── Duck typing
│   └── Operator overloading
├── Abstraction
│   ├── Abstract class
│   └── Abstract method
├── Object relationships
│   ├── Association
│   ├── Aggregation
│   └── Composition
└── Advanced concepts
    ├── MRO
    ├── Magic methods
    ├── Dataclass
    ├── Shallow/deep copy
    └── __new__ and __init__
```

You said:

OOP ├── Class and Object ├── Constructor and self ├── Attributes │ ├── Instance variable │ └── Class variable ├── Methods │ ├── Instance method │ ├── Class method │ └── Static method ├── Encapsulation │ ├── Public │ ├── Protected │ ├── Private │ └── Property ├── Inheritance │ ├── Single │ ├── Multilevel │ ├── Multiple │ ├── Hierarchical │ └── Hybrid ├── Polymorphism │ ├── Overriding │ ├── Duck typing │ └── Operator overloading ├── Abstraction │

├── Abstract class │ └── Abstract method ├── Object relationships │ ├── Association │ ├── Aggregation │ └── Composition └── Advanced concepts ├── MRO ├── Magic methods ├── Dataclass ├── Shallow/deep copy └── **new** and **init** real life example to connect this topic and code also for interview and all concept

ChatGPT said:

Complete Python OOP with Real-Life Examples and Interview Code

We will connect almost every concept with an **Employee Management System**, because it is easy to explain in interviews.

1. Class and Object

Very easy definition

- A **class** is a blueprint or design.
- An **object** is a real item created from that blueprint.
- An **instance** is another name for an object.

Real-life example

- Employee form/design → class
- Aamir, Rahul and Priya → objects

```
class Employee:
    def work(self):
        print("Employee is working")


employee1 = Employee()
employee2 = Employee()

employee1.work()
employee2.work()
```

Here:

- Employee is a class.
- employee1 and employee2 are objects.

- work() is a method.

Object with data

```
class Employee:
    def __init__(self, name, role):
        self.name = name
        self.role = role

    def display(self):
        print("Name:", self.name)
        print("Role:", self.role)


employee1 = Employee("Aamir", "AI/ML Engineer")
employee2 = Employee("Rahul", "Data Analyst")

employee1.display()
employee2.display()
```

Output:

```
Name: Aamir
Role: AI/ML Engineer
Name: Rahul
Role: Data Analyst
```

Interview answer

> A class is a blueprint containing attributes and methods. An object is a real instance created from that class.

2. Constructor and self

Constructor

A constructor is a special method that runs automatically when an object is created.

Python uses __init__() to initialize an object.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary


employee = Employee("Aamir", 50000)

print(employee.name)
print(employee.salary)
```

Dry run

When Python executes:

```
employee = Employee("Aamir", 50000)
```

The following happens:

1. Python creates a new object.
2. self points to that new object.
3. "Aamir" goes into name.
4. 50000 goes into salary.
5. self.name = name saves "Aamir" in the object.
6. self.salary = salary saves 50000.

What is self?

self represents the current object.

```
class Employee:
    def __init__(self, name):
        self.name = name

    def show(self):
        print(self.name)


employee1 = Employee("Aamir")
employee2 = Employee("Rahul")

employee1.show()
employee2.show()
```

When you write:

```
employee1.show()
```

Python internally works approximately like:

```
Employee.show(employee1)
```

Therefore, self receives employee1.

Important points

- self is not a Python keyword.
- We could technically use another name.
- Using self is the standard professional convention.
- self must be the first parameter of an instance method.

Interview answer

> self refers to the current object. It is used to access the current object's variables and methods.

3. Types of Constructors

Python does not support multiple __init__() methods in the traditional Java style.

3.1 Default constructor

```
class Employee:
    def __init__(self):
        self.name = "Unknown"


employee = Employee()
print(employee.name)
```

3.2 Parameterized constructor

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary


employee = Employee("Aamir", 50000)
```

3.3 Constructor with default values

```
class Employee:
    def __init__(self, name="Unknown", salary=0):
        self.name = name
        self.salary = salary


employee1 = Employee()
employee2 = Employee("Aamir", 50000)

print(employee1.name, employee1.salary)
print(employee2.name, employee2.salary)
```

3.4 Alternative constructor

A class method can create an object from a different data format.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    @classmethod
    def from_string(cls, data):
        name, salary = data.split("-")
        return cls(name, int(salary))


employee = Employee.from_string("Aamir-50000")

print(employee.name)
print(employee.salary)
```

4. Attributes

An attribute is a variable belonging to a class or object.

Two important types are:

1. Instance variable
2. Class variable

4.1 Instance variable

An instance variable belongs to one particular object.

It is generally created using self.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary


employee1 = Employee("Aamir", 50000)
employee2 = Employee("Rahul", 60000)

print(employee1.name, employee1.salary)
print(employee2.name, employee2.salary)
```

Every employee has different data.

4.2 Class variable

A class variable belongs to the class and is shared by its objects.

```
class Employee:
    company = "TechNova"

    def __init__(self, name):
        self.name = name


employee1 = Employee("Aamir")
employee2 = Employee("Rahul")

print(employee1.company)
print(employee2.company)
print(Employee.company)
```

Output:

```
TechNova
TechNova
TechNova
```

Changing a class variable

```
Employee.company = "Google"

print(employee1.company)
print(employee2.company)
```

Output:

```
Google
Google
```

Important interview case

```
class Employee:
    company = "TechNova"


employee1 = Employee()
employee2 = Employee()

employee1.company = "Google"

print(employee1.company)
print(employee2.company)
print(Employee.company)
```

Output:

```
Google
TechNova
TechNova
```

`employee1.company = "Google"` creates a new instance variable for `employee1`. It does not change the original class variable.

Comparison

Instance variable	Class variable
Belongs to an object	Belongs to a class
Usually created using `self`	Created directly inside class
Different for each object	Shared by objects
Example: employee name	Example: company name

5. Types of Methods

Python has three main method types:

1. Instance method
2. Class method
3. Static method

5.1 Instance method

An instance method works with object-specific data.

It receives self.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print(self.name, self.salary)

    def increase_salary(self, amount):
        self.salary += amount


employee = Employee("Aamir", 50000)

employee.increase_salary(5000)
employee.display()
```

Output:

```
Aamir 55000
```

5.2 Class method

A class method works with class-level data.

It receives cls and uses the @classmethod decorator.

```
class Employee:
    company = "TechNova"

    @classmethod
    def change_company(cls, new_company):
        cls.company = new_company


Employee.change_company("Google")

print(Employee.company)
```

Output:

```
Google
```

Counting objects with a class variable

```
class Employee:
    employee_count = 0

    def __init__(self, name):
        self.name = name
        Employee.employee_count += 1

    @classmethod
    def total_employees(cls):
        return cls.employee_count


employee1 = Employee("Aamir")
employee2 = Employee("Rahul")
employee3 = Employee("Priya")

print(Employee.total_employees())
```

Output:

```
3
```

5.3 Static method

A static method is a utility function connected to a class.

It does not automatically receive `self` or `cls`.

```
class Employee:
    @staticmethod
    def is_valid_email(email):
        return "@" in email and "." in email


print(Employee.is_valid_email("aamir@gmail.com"))
print(Employee.is_valid_email("aamirgmail"))
```

Output:

```
True
False
```

Method comparison

Method	First parameter	Main purpose
Instance method	self	Works with object data
Class method	cls	Works with class data
Static method	Nothing compulsory	Utility/helper work

Interview answer

> An instance method accesses object data using self. A class method accesses class data using cls. A static method does not directly depend on object or class data.

6. Encapsulation

Very easy definition

Encapsulation means:

- Keeping data and methods together inside a class.
- Controlling how internal data is accessed and changed.

Real-life example

A bank does not allow customers to modify their account balance directly. Customers must use deposit() or withdraw().

Python has three naming levels:

1. Public
2. Protected
3. Private

6.1 Public member

A public member can be accessed from anywhere.

```
class Employee:
    def __init__(self, name):
        self.name = name


employee = Employee("Aamir")

print(employee.name)

employee.name = "Rahul"
print(employee.name)
```

6.2 Protected member

A protected member starts with one underscore `_`.

```
class Employee:
    def __init__(self, name, department):
        self.name = name
        self._department = department


employee = Employee("Aamir", "AI Department")

print(employee._department)
```

Python allows access, but `_department` tells developers:

> This member is meant for internal class and subclass use.

Protected access is a convention, not strict security.

6.3 Private member

A private member starts with two underscores `__`.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary

    def show_salary(self):
        print("Salary:", self.__salary)


employee = Employee("Aamir", 50000)

employee.show_salary()
```

The following normally produces an error:

```
print(employee.__salary)
```

Python internally changes `__salary` to a name similar to:

```
_Employee__salary
```

This process is called **name mangling.**

```
print(employee._Employee__salary)
```

It can still be accessed this way, but it is not recommended.

Interview point

> Python does not have completely strict private variables. Double underscores provide name mangling and help prevent accidental direct access.

6.4 Getter and setter

- Getter reads private data.

- Setter safely changes private data.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    def get_salary(self):
        return self.__salary

    def set_salary(self, new_salary):
        if new_salary < 0:
            print("Salary cannot be negative")
        else:
            self.__salary = new_salary


employee = Employee(50000)

print(employee.get_salary())

employee.set_salary(60000)
print(employee.get_salary())

employee.set_salary(-1000)
```

6.5 Property

@property provides a cleaner way to create getters and setters.

```
class Employee:
    def __init__(self, salary):
        self.__salary = 0
        self.salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, amount):
        if amount < 0:
            raise ValueError("Salary cannot be negative")

        self.__salary = amount


employee = Employee(50000)

print(employee.salary)

employee.salary = 60000
print(employee.salary)
```

Although this looks like normal variable access:

```
employee.salary
```

Python is calling the getter method.

Similarly:

```
employee.salary = 60000
```

calls the setter method.

---

7. Inheritance

Very easy definition

Inheritance means that a child class receives features from a parent class.

Real-life example

Every developer is an employee. Therefore:

```
Developer is an Employee
```

Inheritance represents an **is-a relationship**.

Benefits

- Code reuse
- Less duplicate code
- Easy maintenance
- Easy extension
- Better organization

8. Single Inheritance

One child class inherits from one parent class.

```
class Employee:
    def login(self):
        print("Employee logged in")


class Developer(Employee):
    def write_code(self):
        print("Developer is writing Python code")


developer = Developer()

developer.login()
developer.write_code()
```

Relationship:

```
Employee → Developer
```

9. Multilevel Inheritance

A class inherits through multiple levels.

```
class Person:
    def introduce(self):
        print("I am a person")


class Employee(Person):
    def work(self):
        print("I am an employee")


class Developer(Employee):
    def write_code(self):
        print("I write Python code")


developer = Developer()

developer.introduce()
developer.work()
developer.write_code()
```

Relationship:

```
Person → Employee → Developer
```

---

10. Multiple Inheritance

One child class inherits from multiple parents.

```
class Programmer:
    def code(self):
        print("Writing code")


class Manager:
    def manage_team(self):
        print("Managing the team")


class TechLead(Programmer, Manager):
    def plan_project(self):
        print("Planning the project")


lead = TechLead()

lead.code()
lead.manage_team()
lead.plan_project()
```

Relationship:

```
Programmer ─┐
            ├── TechLead
Manager ────┘
```

---

11. Hierarchical Inheritance

Multiple child classes inherit from one parent.

```
class Employee:
    def login(self):
        print("Employee logged in")


class Developer(Employee):
    def work(self):
        print("Developer writes code")


class Tester(Employee):
    def work(self):
        print("Tester tests software")


class DataAnalyst(Employee):
    def work(self):
        print("Data Analyst analyzes data")


developer = Developer()
tester = Tester()
analyst = DataAnalyst()

developer.login()
developer.work()

tester.login()
tester.work()

analyst.login()
analyst.work()
```

Relationship:

```
          Employee
        /    |     \
 Developer   Tester   DataAnalyst
```

12. Hybrid Inheritance

Hybrid inheritance is a combination of two or more inheritance types.

```
class Person:
    def introduce(self):
        print("I am a person")


class Employee(Person):
    def login(self):
        print("Employee logged in")


class TechnicalSkill:
    def technical_work(self):
        print("Performing technical work")


class Developer(Employee, TechnicalSkill):
    def write_code(self):
        print("Writing Python code")


class TeamLead(Developer):
    def manage_team(self):
        print("Managing development team")


lead = TeamLead()

lead.introduce()
lead.login()
lead.technical_work()
lead.write_code()
lead.manage_team()
```

This combines:

- Multilevel inheritance
- Multiple inheritance

---

13. super()

super() is used to access parent-class methods.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print("Name:", self.name)
        print("Salary:", self.salary)


class Developer(Employee):
    def __init__(self, name, salary, language):
        super().__init__(name, salary)
        self.language = language

    def display(self):
        super().display()
        print("Language:", self.language)


developer = Developer("Aamir", 50000, "Python")
developer.display()
```

Output:

```
Name: Aamir
Salary: 50000
Language: Python
```

Without this:

```
super().__init__(name, salary)
```

the parent initialization would not automatically execute.

Interview answer

> super() allows a child class to call parent-class methods without directly writing the parent class name. It is especially useful in multiple inheritance.

14. Polymorphism

Very easy definition

Polymorphism means the same method name can perform different actions for different objects.

- Poly means many.
- Morph means forms.
- Polymorphism means many forms.

15. Method Overriding

Overriding happens when a child class writes its own version of a parent method.

```
class Employee:
    def work(self):
        print("Employee is working")


class Developer(Employee):
    def work(self):
        print("Developer is writing code")


class Tester(Employee):
    def work(self):
        print("Tester is testing software")


employees = [Developer(), Tester()]

for employee in employees:
    employee.work()
```

Output:

```
Developer is writing code
Tester is testing software
```

Calling both parent and child methods

```
class Employee:
    def work(self):
        print("Completing general employee work")


class Developer(Employee):
    def work(self):
        super().work()
        print("Writing Python code")


developer = Developer()
developer.work()
```

Output:

```
Completing general employee work
Writing Python code
```

16. Method Overloading

Method overloading means methods have the same name but different parameters.

Python does not support traditional method overloading.

This does not work as expected:

```
class Calculator:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c
```

The second `add()` replaces the first one.

Using default parameters

```
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c


calculator = Calculator()

print(calculator.add(10, 20))
print(calculator.add(10, 20, 30))
```

Output:

```
30
60
```

Using *args

```
class Calculator:
    def add(self, *numbers):
        return sum(numbers)


calculator = Calculator()

print(calculator.add(10, 20))
print(calculator.add(10, 20, 30, 40))
```

17. Duck Typing

Very easy definition

Duck typing means Python cares about an object's behaviour, not its exact class.

> If an object has the required method, Python can use it.

```
class Employee:
    def work(self):
        print("Employee is working")


class Freelancer:
    def work(self):
        print("Freelancer is working remotely")


class Robot:
    def work(self):
        print("Robot is performing automated work")


def start_work(worker):
    worker.work()


start_work(Employee())
start_work(Freelancer())
start_work(Robot())
```

start_work() does not check the object's class. It only expects a work() method.

Interview answer

> Duck typing means that if an object provides the required behaviour or method, Python can use it without checking its exact type.

18. Operator Overloading

Operator overloading defines how operators work with custom objects.

Real-life example

We want to add the salaries of two employees using +.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __add__(self, other):
        return self.salary + other.salary


employee1 = Employee("Aamir", 50000)
employee2 = Employee("Rahul", 60000)

print(employee1 + employee2)
```

Output:

```
110000
```

Python internally converts:

```
employee1 + employee2
```

into:

```
employee1.__add__(employee2)
```

Equality overloading

```
class Employee:
    def __init__(self, employee_id, name):
        self.employee_id = employee_id
        self.name = name

    def __eq__(self, other):
        if not isinstance(other, Employee):
            return NotImplemented

        return self.employee_id == other.employee_id


employee1 = Employee(101, "Aamir")
employee2 = Employee(101, "Aamir Azad")
employee3 = Employee(102, "Rahul")

print(employee1 == employee2)
print(employee1 == employee3)
```

Output:

```
True
False
```

19. Abstraction

Very easy definition

Abstraction means hiding complicated internal details and showing only the necessary features.

Real-life example

When an employee clicks “Generate Salary,” the employee does not need to know every internal salary calculation.

Python provides abstraction through:

- ABC
- @abstractmethod

ABC means **Abstract Base Class**.

20. Abstract Class and Abstract Method

```
from abc import ABC, abstractmethod


class Employee(ABC):

    @abstractmethod
    def calculate_salary(self):
        pass


class FullTimeEmployee(Employee):
    def __init__(self, monthly_salary):
        self.monthly_salary = monthly_salary

    def calculate_salary(self):
        return self.monthly_salary


class Freelancer(Employee):
    def __init__(self, hours, rate):
        self.hours = hours
        self.rate = rate

    def calculate_salary(self):
        return self.hours * self.rate


full_time = FullTimeEmployee(50000)
freelancer = Freelancer(100, 500)

print(full_time.calculate_salary())
print(freelancer.calculate_salary())
```

Output:

```
50000
50000
```

This produces an error:

```
employee = Employee()
```

Why?

Because Employee contains an incomplete abstract method.

Important terms

Term	Easy meaning
Abstract class	An incomplete parent class used as a design
Abstract method	A method the child class must implement
Concrete class	A complete class whose object can be created
Implementation	The actual code inside a method

Interview answer

> An abstract class defines common rules for child classes. An abstract method has no complete implementation in the parent and must be implemented by concrete child classes.

21. Encapsulation vs Abstraction

Encapsulation	Abstraction
Protects internal data	Hides complicated implementation
Controls data access	Shows only necessary features
Uses private members and properties	Uses abstract classes and methods
Example: protecting salary	Example: hiding salary calculation

22. Object Relationships

Object relationships explain how different objects work together.

Important relationships:

1. Association
2. Aggregation
3. Composition

23. Association

Very easy definition

Association means two independent objects communicate or work together.

Real-life example

An employee works on a project. Both employee and project can exist separately.

```
class Employee:
    def __init__(self, name):
        self.name = name

    def work_on(self, project):
        print(f"{self.name} is working on {project.title}")


class Project:
    def __init__(self, title):
        self.title = title


employee = Employee("Aamir")
project = Project("Crop Yield Prediction")

employee.work_on(project)
```

Both objects are independently created.

24. Aggregation

Very easy definition

Aggregation is a weak **has-a relationship**.

One object contains other independently created objects.

Real-life example

A department has employees. Employees can still exist if the department is removed.

```
class Employee:
    def __init__(self, name):
        self.name = name


class Department:
    def __init__(self, name, employees):
        self.name = name
        self.employees = employees

    def display_employees(self):
        for employee in self.employees:
            print(employee.name)


employee1 = Employee("Aamir")
employee2 = Employee("Rahul")

it_department = Department(
    "Information Technology",
    [employee1, employee2]
)

it_department.display_employees()
```

The employees are created outside the department and then given to it.

25. Composition

Very easy definition

Composition is a strong **has-a relationship**.

The parent object creates and strongly owns the child object.

Real-life example

An employee record creates an ID card.

```
class IDCard:
    def __init__(self, employee_id):
        self.employee_id = employee_id

    def display(self):
        print("ID Card:", self.employee_id)


class Employee:
    def __init__(self, name, employee_id):
        self.name = name
        self.id_card = IDCard(employee_id)

    def display(self):
        print("Employee:", self.name)
        self.id_card.display()


employee = Employee("Aamir", 101)
employee.display()
```

Here, Employee creates its own IDCard.

Relationship comparison

Relationship	Meaning	Example
Association	Objects work together	Employee works on Project
Aggregation	Weak has-a relationship	Department has Employees
Composition	Strong has-a relationship	Employee record has ID Card
Inheritance	Is-a relationship	Developer is an Employee

26. MRO

MRO means **Method Resolution Order**.

It is the order Python follows when searching for a method in inheritance.

```
class A:
    def show(self):
        print("Class A")


class B(A):
    def show(self):
        print("Class B")


class C(A):
    def show(self):
        print("Class C")


class D(B, C):
    pass


obj = D()
obj.show()

print(D.mro())
```

Output:

```
Class B
[D, B, C, A, object]
```

Python searches:

```
D → B → C → A → object
```

MRO with super()

```
class A:
    def show(self):
        print("A")


class B(A):
    def show(self):
        print("B")
        super().show()


class C(A):
    def show(self):
        print("C")
        super().show()


class D(B, C):
    pass


obj = D()
obj.show()
```

Output:

```
B
C
A
```

This happens because the MRO is:

```
D → B → C → A → object
```

Inside B, super() does not simply mean A. It means the next class in the current MRO, which is C.

Interview answer

> MRO is the sequence Python follows to search for methods in multiple inheritance. We can inspect it using ClassName.mro() or ClassName.__mro__.

27. Magic or Dunder Methods

Dunder means **double underscore.**

Examples:

```
__init__
__str__
__repr__
__len__
__add__
__eq__
```

Magic methods allow custom objects to work with built-in Python operations.

__str__()

Provides user-friendly output.

```
class Employee:
    def __init__(self, name, role):
        self.name = name
        self.role = role

    def __str__(self):
        return f"{self.name} works as {self.role}"


employee = Employee("Aamir", "AI/ML Engineer")

print(employee)
```

Output:

```
Aamir works as AI/ML Engineer
```

__repr__()

Provides detailed developer-friendly output.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __repr__(self):
        return f"Employee(name='{self.name}', salary={self.salary})"


employee = Employee("Aamir", 50000)

print(repr(employee))
```

__len__()

Controls the len() function.

```
class Department:
    def __init__(self, employees):
        self.employees = employees

    def __len__(self):
        return len(self.employees)


department = Department(["Aamir", "Rahul", "Priya"])

print(len(department))
```

Output:

```
3
```

__call__()

Makes an object callable like a function.

```
class SalaryCalculator:
    def __call__(self, basic_salary, bonus):
        return basic_salary + bonus


calculator = SalaryCalculator()

print(calculator(50000, 5000))
```

Output:

```
55000
```

Common magic methods

Method	Used for
`__new__()`	Creating an object
`__init__()`	Initializing an object
`__str__()`	User-friendly text
`__repr__()`	Developer-friendly text
`__len__()`	`len(object)`
`__add__()`	`object1 + object2`
`__sub__()`	`object1 - object2`
`__eq__()`	`object1 == object2`
`__lt__()`	`object1 < object2`
`__getitem__()`	`object[index]`
`__call__()`	`object()`
`__enter__()`	Beginning of a `with` block
`__exit__()`	End of a `with` block

28. Dataclass

Very easy definition

A dataclass is useful when a class mainly stores data.

It automatically generates methods such as:

- __init__()
- __repr__()
- __eq__()

```
from dataclasses import dataclass


@dataclass
class Employee:
    employee_id: int
    name: str
    salary: float


employee1 = Employee(101, "Aamir", 50000)
employee2 = Employee(101, "Aamir", 50000)

print(employee1)
print(employee1 == employee2)
```

Output:

```
Employee(employee_id=101, name='Aamir', salary=50000)
True
```

Without dataclass

We would need to manually write:

```
class Employee:
    def __init__(self, employee_id, name, salary):
        self.employee_id = employee_id
        self.name = name
        self.salary = salary
```

Immutable dataclass

Immutable means that the data cannot be changed after object creation.

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Address:
    city: str
    state: str


address = Address("Jaipur", "Rajasthan")

print(address)
```

This gives an error:

```
address.city = "Delhi"
```

Because frozen=True makes it read-only.

29. Shallow Copy and Deep Copy

Suppose an employee object contains a list of skills.

Shallow copy

A shallow copy creates a new outer object, but nested objects are shared.

```
import copy


class Employee:
    def __init__(self, name, skills):
        self.name = name
        self.skills = skills


employee1 = Employee("Aamir", ["Python", "SQL"])
employee2 = copy.copy(employee1)

employee1.skills.append("Machine Learning")

print(employee1.skills)
print(employee2.skills)
```

Output:

```
['Python', 'SQL', 'Machine Learning']
['Python', 'SQL', 'Machine Learning']
```

Both objects share the same nested skills list.

Deep copy

A deep copy creates independent copies of nested objects.

```
import copy


class Employee:
    def __init__(self, name, skills):
        self.name = name
        self.skills = skills


employee1 = Employee("Aamir", ["Python", "SQL"])
employee2 = copy.deepcopy(employee1)

employee1.skills.append("Machine Learning")

print(employee1.skills)
print(employee2.skills)
```

Output:

```
['Python', 'SQL', 'Machine Learning']
['Python', 'SQL']
```

Comparison

Shallow copy	Deep copy
Copies outer object	Copies outer and nested objects
Nested objects may be shared	Nested objects are independent
Uses `copy.copy()`	Uses `copy.deepcopy()`
Faster	Usually consumes more memory

30. `__new__()` and `__init__()`

`__new__()`

`__new__()` creates and returns a new object.

`__init__()`

`__init__()` initializes the already-created object.

```
class Employee:
    def __new__(cls, *args, **kwargs):
        print("1. __new__ is creating the object")
        obj = super().__new__(cls)
        return obj

    def __init__(self, name):
        print("2. __init__ is initializing the object")
        self.name = name


employee = Employee("Aamir")

print(employee.name)
```

Output:

```
1. __new__ is creating the object
2. __init__ is initializing the object
Aamir
```

Comparison

`__new__()`	`__init__()`
Creates an object	Initializes an object
Executes first	Executes after `__new__()`
Receives `cls`	Receives `self`
Must return an object	Should return `None`
Rarely overridden	Commonly used

Important interview point

> Technically, `__new__()` is the constructor and `__init__()` is the initializer. However, `__init__()` is commonly called the constructor in normal interviews.

31. Complete Connected Employee Management Project

This example combines almost all major OOP concepts.

```
from abc import ABC, abstractmethod
from dataclasses import dataclass


# Dataclass
@dataclass(frozen=True)
class Address:
    city: str
    state: str


# Composition: Employee creates and owns an IDCard
class IDCard:
    def __init__(self, employee_id):
        self.employee_id = employee_id

    def __str__(self):
        return f"ID-{self.employee_id}"


# Association: Project exists independently
class Project:
    def __init__(self, title):
        self.title = title

    def __str__(self):
        return self.title


# Abstraction
class Employee(ABC):

    # Class variables
    company_name = "TechNova"
    employee_count = 0

    def __init__(self, employee_id, name, email, salary, address):
        # Public attributes
        self.employee_id = employee_id
        self.name = name
        self.email = email

        # Private attribute
        self.__salary = 0
        self.salary = salary

        # Aggregated object
        self.address = address

        # Composed object
```

```
        self.id_card = IDCard(employee_id)

        # Protected attribute
        self._projects =

        Employee.employee_count += 1

    # Property getter
    @property
    def salary(self):
        return self.__salary

    # Property setter
    @salary.setter
    def salary(self, amount):
        if amount < 0:
            raise ValueError("Salary cannot be negative")

        self.__salary = amount

    # Instance method
    def assign_project(self, project):
        self._projects.append(project)

    # Class method
    @classmethod
    def change_company(cls, new_company):
        cls.company_name = new_company

    # Static method
    @staticmethod
    def validate_email(email):
        return "@" in email and "." in email

    # Abstract method
    @abstractmethod
    def calculate_monthly_pay(self):
        pass

    # Abstract method
    @abstractmethod
    def work(self):
        pass

    # Magic method
    def __str__(self):
        return (
            f"{self.name} | {self.__class__.__name__} | "
            f"{self.id_card}"
        )
```

```
    # Developer representation
    def __repr__(self):
        return (
            f"{self.__class__.__name__}("
            f"employee_id={self.employee_id}, "
            f"name='{self.name}')"
        )

    # Operator overloading
    def __add__(self, other):
        if not isinstance(other, Employee):
            return NotImplemented

        return (
            self.calculate_monthly_pay()
            + other.calculate_monthly_pay()
        )

    # Equality based on employee ID
    def __eq__(self, other):
        if not isinstance(other, Employee):
            return NotImplemented

        return self.employee_id == other.employee_id

    # Less-than comparison based on salary
    def __lt__(self, other):
        if not isinstance(other, Employee):
            return NotImplemented

        return (
            self.calculate_monthly_pay()
            < other.calculate_monthly_pay()
        )


# Inheritance
class FullTimeEmployee(Employee):
    def __init__(
        self,
        employee_id,
        name,
        email,
        salary,
        address,
        bonus=0
    ):
        super().__init__(
            employee_id,
            name,
            email,
```

```
            salary,
            address
        )
        self.bonus = bonus

    def calculate_monthly_pay(self):
        return self.salary + self.bonus

    def work(self):
        print(f"{self.name} works full-time")


# Multilevel inheritance
class Developer(FullTimeEmployee):
    def __init__(
        self,
        employee_id,
        name,
        email,
        salary,
        address,
        language,
        bonus=0
    ):
        super().__init__(
            employee_id,
            name,
            email,
            salary,
            address,
            bonus
        )
        self.language = language

    # Method overriding
    def work(self):
        print(
            f"{self.name} develops applications "
            f"using {self.language}"
        )


class Freelancer(Employee):
    def __init__(
        self,
        employee_id,
        name,
        email,
        address,
        working_hours,
        hourly_rate
```

```
    ):
        super().__init__(
            employee_id,
            name,
            email,
            0,
            address
        )
        self.working_hours = working_hours
        self.hourly_rate = hourly_rate

    def calculate_monthly_pay(self):
        return self.working_hours * self.hourly_rate

    def work(self):
        print(f"{self.name} works remotely as a freelancer")


# Aggregation: Department stores independent employee objects
class Department:
    def __init__(self, name):
        self.name = name
        self.employees =

    def add_employee(self, employee):
        self.employees.append(employee)

    def __len__(self):
        return len(self.employees)

    def display_employees(self):
        print(f"\nDepartment: {self.name}")

        for employee in self.employees:
            print(employee)


# Duck typing
class PayrollService:
    @staticmethod
    def generate_salary(worker):
        print(
            f"{worker.name}'s salary: "
            f"₹{worker.calculate_monthly_pay()}"
        )


# Creating independent address objects
address1 = Address("Jaipur", "Rajasthan")
address2 = Address("Delhi", "Delhi")
```

```
# Creating project
ml_project = Project("Crop Yield Prediction")

# Creating employee objects
developer = Developer(
    employee_id=101,
    name="Aamir",
    email="aamir@gmail.com",
    salary=50000,
    address=address1,
    language="Python",
    bonus=5000
)

freelancer = Freelancer(
    employee_id=102,
    name="Rahul",
    email="rahul@gmail.com",
    address=address2,
    working_hours=100,
    hourly_rate=500
)

# Association
developer.assign_project(ml_project)

# Aggregation
ai_department = Department("AI and Machine Learning")
ai_department.add_employee(developer)
ai_department.add_employee(freelancer)

# Polymorphism
for employee in ai_department.employees:
    employee.work()
    PayrollService.generate_salary(employee)

# Magic method: __len__
print("Total department employees:", len(ai_department))

# Magic method: __str__
ai_department.display_employees()

# Operator overloading: __add__
print(
    "Combined monthly payment:",
    developer + freelancer
)

# Class variable
print("Company:", Employee.company_name)
print("Total created employees:", Employee.employee_count)
```

```
# Static method
print(
    "Valid email:",
    Employee.validate_email("aamir@gmail.com")
)

# Class method
Employee.change_company("OpenAI Tech")
print("New company:", Employee.company_name)
```

Concepts used in this project

Concept	Project example
Class	Employee, Developer, Project
Object	developer, freelancer
Constructor	__init__()
self	Current employee object
Instance variable	name, email, salary
Class variable	company_name, employee_count
Instance method	assign_project()
Class method	change_company()
Static method	validate_email()
Public	name
Protected	_projects
Private	__salary
Property	salary getter and setter
Inheritance	FullTimeEmployee(Employee)
Multilevel inheritance	Developer(FullTimeEmployee)
Overriding	Developer.work()

Concept	Project example
Polymorphism	Different work() implementations
Duck typing	PayrollService.generate_salary()
Abstraction	Employee(ABC)
Abstract method	calculate_monthly_pay()
Association	Employee and Project
Aggregation	Department and Employees
Composition	Employee and IDCard
Dataclass	Address
Magic methods	__str__, __len__, __add__

32. Important Interview Output Questions

Question 1: Class vs instance variable

```
class Demo:
    value = 10


a = Demo()
b = Demo()

a.value = 20
Demo.value = 30

print(a.value)
print(b.value)
```

Output:

```
20
30
```

Reason:

- a.value = 20 creates an instance variable.
- b still reads the class variable.
- The class variable later becomes 30.

Question 2: Shared mutable class variable

```
class Employee:
    skills =


employee1 = Employee()
employee2 = Employee()

employee1.skills.append("Python")

print(employee1.skills)
print(employee2.skills)
```

Output:

```
['Python']
['Python']
```

Both objects share the same class-level list.

Correct version

```
class Employee:
    def __init__(self):
        self.skills =
```

Now every employee receives a separate list.

Question 3: Overriding

```
class Parent:
    def show(self):
        print("Parent")


class Child(Parent):
    def show(self):
        print("Child")


obj = Child()
obj.show()
```

Output:

```
Child
```

The child method overrides the parent method.

Question 4: MRO

```
class A:
    def show(self):
        print("A")


class B(A):
    def show(self):
        print("B")


class C(A):
    def show(self):
        print("C")


class D(C, B):
    pass


D().show()
```

Output:

```
C
```

MRO:

```
D → C → B → A → object
```

Question 5: Object equality

```
class Employee:
    def __init__(self, name):
        self.name = name


employee1 = Employee("Aamir")
employee2 = Employee("Aamir")

print(employee1 == employee2)
```

Output:

```
False
```

The objects contain the same data but are two different objects.

To compare their data, implement `__eq__()`.

33. Most Important Interview Questions

1. What is OOP?

OOP is a programming approach that organizes programs using classes and objects. It improves code reuse, organization and maintenance.

2. What are the four pillars of OOP?

Encapsulation, inheritance, polymorphism and abstraction.

3. What is an object?

An object is an instance of a class containing its own data and behaviour.

4. What is self?

self represents the current object.

5. What is cls?

cls represents the current class inside a class method.

6. What is inheritance?

Inheritance allows a child class to reuse and extend a parent class.

7. What is polymorphism?

Polymorphism allows the same method name to behave differently for different objects.

8. What is method overriding?

A child class provides its own version of an inherited parent method.

9. Does Python support method overloading?

Not in the traditional form. It can be achieved using default parameters, *args or dispatch techniques.

10. What is abstraction?

Abstraction hides complicated implementation and shows only necessary operations.

11. Can we create an abstract-class object?

Not when it contains unimplemented abstract methods.

12. What is encapsulation?

Encapsulation groups data with methods and controls how internal data is accessed.

13. Are private variables completely private in Python?

No. Python uses name mangling, but the value can technically still be accessed.

14. What is MRO?

MRO is the order Python follows to search classes for a method.

15. What is duck typing?

Python uses an object if it provides the required method, regardless of its exact class.

16. What is composition?

Composition is a strong has-a relationship in which one object creates or owns another object.

17. What is aggregation?

Aggregation is a weak has-a relationship involving independently created objects.

18. What is a dataclass?

A dataclass automatically creates methods such as `__init__()`, `__repr__()` and `__eq__()` for data-holding classes.

19. What is the difference between `is` and `==`?

- `is` checks whether variables point to the same object.
- `==` checks whether their values are equal.

20. What is the difference between `__new__()` and `__init__()`?

`__new__()` creates an object, while `__init__()` initializes it.

---

34. Difficult Words in Easy Language

Difficult word	Easy meaning
Blueprint	A plan used to create something
Instance	An object created from a class
Attribute	A variable inside a class or object
Method	A function inside a class
Initialize	Give starting values
Instantiate	Create an object
Encapsulation	Protect and control data
Inheritance	Receive features from a parent
Polymorphism	Same action, different behaviour
Abstraction	Hide unnecessary details
Override	Rewrite a parent method

Difficult word	Easy meaning
Implementation	Actual working code
Immutable	Cannot be changed
Mutable	Can be changed
Association	Objects work together
Aggregation	Weak has-a relationship
Composition	Strong has-a relationship
Utility method	Helper method
Name mangling	Python changes a private variable's name
MRO	Method-searching order
Dunder	Double underscore
Concrete class	Complete class whose object can be created
Instantiation	Process of creating an object
Behaviour	What an object can do
State	Current data of an object

You said:

oops all interview questions and with very easy word ans

ChatGPT said:

Python OOP Interview Questions and Answers

In interviews, answer using this simple pattern:

> **Definition → real-life example → benefit**

Example:

> Inheritance allows a child class to reuse features of a parent class. For example, a Developer is an Employee. It reduces duplicate code.

A. Basic OOP Questions

1. What is OOP?

OOP stands for **Object-Oriented Programming**. It organizes a program using classes and objects.

Example: An employee management application can contain Employee, Developer and Manager classes.

2. Why do we use OOP?

We use OOP to:

- Reuse code
- Reduce duplicate code
- Protect data
- Organize large programs
- Make code easier to maintain
- Represent real-world things

3. What are the four pillars of OOP?

The four pillars are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

4. What is a class?

A class is a blueprint used to create objects.

```
class Employee:
    pass
```

Employee is a class.

5. What is an object?

An object is a real instance created from a class.

```
class Employee:
    pass

employee1 = Employee()
```

employee1 is an object.

6. What is an instance?

An instance is another name for an object created from a class.

```
employee1 = Employee()
```

employee1 is an instance of Employee.

7. What is the difference between class and object?

Class	Object
Blueprint	Real instance
Logical design	Uses memory
Created once	Many objects can be created
Example: Employee	Example: employee1

8. What is an attribute?

An attribute is a variable belonging to a class or object.

```
class Employee:
    def __init__(self, name):
        self.name = name
```

name is an attribute.

9. What is a method?

A method is a function written inside a class.

```
class Employee:
    def work(self):
        print("Employee is working")
```

work() is a method.

10. What is an object's state?

State means the current data stored in an object.

For an employee, state can be:

- Name
- Salary
- Department
- Employee ID

11. What is an object's behaviour?

Behaviour means what an object can do.

For example:

```
employee.work()
employee.login()
employee.calculate_salary()
```

12. What is object identity?

Identity means an object's unique location in memory.

```
employee = Employee()

print(id(employee))
```

id() returns the object's identity.

13. Is everything an object in Python?

Almost everything in Python is an object, including:

- Numbers
- Strings
- Lists
- Functions
- Classes

```
print(type(10))
print(type("Aamir"))
print(type(Employee))
```

14. Is a class also an object in Python?

Yes. A Python class is normally an object of the type class.

```
class Employee:
    pass

print(type(Employee))
```

Output:

```
<class 'type'>
```

15. What is the base class of all Python classes?

The base class is object.

```
class Employee:
    pass

print(Employee.mro())
```

Output includes:

```
Employee → object
```

B. Constructor and self

16. What is a constructor?

A constructor is a special method that runs during object creation.

In Python, __init__() initializes an object.

```
class Employee:
    def __init__(self, name):
        self.name = name
```

17. What is __init__()?

__init__() is a special method that gives starting values to an object.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

18. Is __init__() the real constructor?

Technically:

- __new__() creates the object.
- __init__() initializes the object.

However, __init__() is commonly called the constructor in interviews.

19. What is self?

self refers to the current object.

```
class Employee:
    def __init__(self, name):
        self.name = name
```

Here, self.name belongs to the current employee object.

20. Is self a Python keyword?

No. self is not a keyword, but it is the standard name used for the current object.

21. Why is self required?

self tells Python which object's data the method should access.

```
employee1.show()
```

Internally, it works approximately like:

```
Employee.show(employee1)
```

22. Can we use another name instead of self?

Yes, but it is not recommended.

```
class Employee:
    def __init__(current_object, name):
        current_object.name = name
```

Always use self in professional code.

23. Can __init__() return a value?

No. __init__() must return None.

This is incorrect:

```
def __init__(self):
    return 10
```

It produces an error.

24. What is a default constructor?

A constructor without required arguments is commonly called a default constructor.

```
class Employee:
    def __init__(self):
        self.name = "Unknown"
```

25. What is a parameterized constructor?

A constructor that receives values is called a parameterized constructor.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

26. Does Python support multiple constructors?

Python does not support multiple __init__() methods directly. The latest definition replaces previous definitions.

We can use:

- Default arguments
- *args
- Class methods

27. What is an alternative constructor?

An alternative constructor is usually a class method that creates an object from another data format.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    @classmethod
    def from_string(cls, data):
        name, salary = data.split("-")
        return cls(name, int(salary))


employee = Employee.from_string("Aamir-50000")
```

28. What is __new__()?

__new__() creates and returns a new object.

```
class Employee:
    def __new__(cls, *args):
        print("Creating object")
        return super().__new__(cls)

    def __init__(self, name):
        print("Initializing object")
        self.name = name
```

__new__() runs before __init__().

29. What is the difference between __new__() and __init__()?

__new__()	__init__()
Creates an object	Initializes an object
Runs first	Runs second
Receives cls	Receives self
Must return an object	Must return None

30. What is a destructor?

__del__() is called when an object is being destroyed.

```
class Employee:
    def __del__(self):
        print("Object is being destroyed")
```

Do not depend on __del__() for important cleanup because its execution time is not guaranteed.

31. Does del object always destroy the object?

No. del removes a reference. The object is destroyed only when no useful references remain and Python cleans it up.

C. Attributes and Methods

32. What is an instance variable?

An instance variable belongs to one object.

```
class Employee:
    def __init__(self, name):
        self.name = name
```

Different objects can have different names.

33. What is a class variable?

A class variable belongs to the class and is shared by its objects.

```
class Employee:
    company = "TechNova"
```

34. Instance variable vs class variable?

Instance variable	Class variable
Belongs to one object	Belongs to class
Usually uses self	Written directly in class
Different for each object	Shared by objects
Example: name	Example: company

35. What happens if an object changes a class variable?

It may create a new instance variable for that object.

```
class Employee:
    company = "TechNova"


e1 = Employee()
e2 = Employee()

e1.company = "Google"

print(e1.company)
print(e2.company)
```

Output:

```
Google
TechNova
```

36. What is an instance method?

An instance method works with object data and receives self.

```
class Employee:
    def display(self):
        print("Employee details")
```

37. What is a class method?

A class method works with class-level data and receives cls.

```
class Employee:
    company = "TechNova"

    @classmethod
    def change_company(cls, company):
        cls.company = company
```

38. What is cls?

cls refers to the current class inside a class method.

It is similar to self, but:

- self represents an object.
- cls represents a class.

---

39. What is a static method?

A static method is a helper method that does not automatically receive self or cls.

```
class Employee:
    @staticmethod
    def validate_email(email):
        return "@" in email
```

---

40. Instance method vs class method vs static method?

Method	Parameter	Purpose
Instance method	self	Object data
Class method	cls	Class data
Static method	Nothing compulsory	Helper logic

---

41. Can a static method access instance variables?

Not directly because it does not receive self. An object must be passed manually if needed.

---

42. Can a class method create an object?

Yes. That is why class methods are often used as alternative constructors.

D. Encapsulation

43. What is encapsulation?

Encapsulation means keeping data and related methods together and controlling access to internal data.

Example: Salary should be changed through a validated method.

---

44. What are access modifiers?

Access modifiers describe how members should be accessed.

Python commonly uses:

- Public: `name`
- Protected: `_name`
- Private: `__name`

Python does not strictly enforce them like Java.

45. What is a public member?

A public member can be accessed from anywhere.

```
self.name = "Aamir"
```

46. What is a protected member?

A protected member starts with one underscore.

```
self._department = "AI"
```

It can still be accessed, but it is intended for internal or subclass use.

47. What is a private member?

A private member starts with two underscores.

```
self.__salary = 50000
```

Python uses name mangling to reduce accidental direct access.

48. Are private variables completely private in Python?

No. They can technically be accessed using their mangled name.

```
employee._Employee__salary
```

However, doing this is not recommended.

49. What is name mangling?

Name mangling means Python changes a double-underscore variable's internal name.

```
__salary
```

becomes approximately:

```
_Employee__salary
```

50. What is a getter?

A getter is a method used to read private data.

```
def get_salary(self):
    return self.__salary
```

51. What is a setter?

A setter is a method used to update private data safely.

```
def set_salary(self, salary):
    if salary >= 0:
        self.__salary = salary
```

52. What is @property?

@property allows a method to be accessed like an attribute.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary
```

Now we can write:

```
print(employee.salary)
```

53. Why use a property?

A property allows us to:

- Validate values
- Protect internal data
- Keep clean attribute-like syntax
- Change internal implementation later

54. How do you make a read-only property?

Create a getter without a setter.

```
class Employee:
    def __init__(self, employee_id):
        self.__employee_id = employee_id

    @property
    def employee_id(self):
        return self.__employee_id
```

55. Encapsulation vs data hiding?

- Encapsulation groups data and methods together.
- Data hiding limits direct access to internal data.

Data hiding is one part of encapsulation.

E. Inheritance

56. What is inheritance?

Inheritance allows a child class to receive and reuse features from a parent class.

```
class Employee:
    def login(self):
        print("Logged in")


class Developer(Employee):
    pass
```

57. Why do we use inheritance?

We use inheritance for:

- Code reuse
- Less duplicate code
- Easy extension
- Better organization
- Polymorphism

58. What is a parent class?

A parent class is the class whose features are inherited.

It is also called:

- Base class
- Superclass

59. What is a child class?

A child class inherits from another class.

It is also called:

- Derived class
- Subclass

60. What are the types of inheritance?

Python supports:

1. Single inheritance
2. Multilevel inheritance
3. Multiple inheritance

4. Hierarchical inheritance
5. Hybrid inheritance

61. What is single inheritance?

One child inherits from one parent.

```
Employee → Developer
```

62. What is multilevel inheritance?

Inheritance happens through multiple levels.

```
Person → Employee → Developer
```

63. What is multiple inheritance?

One child inherits from multiple parents.

```
Programmer ─┐
            ├── TechLead
Manager ────┘
```

64. What is hierarchical inheritance?

Multiple child classes inherit from one parent.

```
          Employee
        /    |    \
Developer Tester Analyst
```

65. What is hybrid inheritance?

Hybrid inheritance combines two or more inheritance types.

For example, it can combine multiple and multilevel inheritance.

66. Does Python support multiple inheritance?

Yes. A Python class can inherit from multiple parent classes.

```
class Child(Parent1, Parent2):
    pass
```

67. What is an is-a relationship?

Inheritance represents an is-a relationship.

Examples:

- Developer is an Employee.
- Dog is an Animal.
- Car is a Vehicle.

68. What is super()?

super() calls the next parent-class method according to the MRO.

```
class Developer(Employee):
    def __init__(self, name, language):
        super().__init__(name)
        self.language = language
```

69. Why should we use super()?

It:

- Reuses parent logic
- Avoids writing the parent class name
- Works properly with multiple inheritance
- Follows MRO

70. Does super() always mean the direct parent?

Not exactly. It means the **next class in the current MRO**.

This is important in multiple inheritance.

71. Are constructors inherited?

The child can use the parent's `__init__()` if it does not define its own.

If the child defines its own `__init__()`, it should call the parent initialization using `super()` when needed.

72. Can a child access parent methods?

Yes. Public and protected parent methods are inherited.

Private members are affected by name mangling and should not be accessed directly.

73. Can a parent access child-specific methods?

Not normally. The parent does not automatically know about features added only by the child.

74. What is `isinstance()`?

`isinstance()` checks whether an object belongs to a class or its parent class.

```
print(isinstance(developer, Developer))
print(isinstance(developer, Employee))
```

75. What is `issubclass()`?

`issubclass()` checks whether one class inherits from another.

```
print(issubclass(Developer, Employee))
```

76. What is MRO?

MRO stands for **Method Resolution Order**. It is the order Python follows when searching for methods.

```
print(Developer.mro())
```

77. Why is MRO needed?

MRO removes confusion when multiple parent classes contain methods with the same name.

78. How can we check MRO?

```
ClassName.mro()
```

or:

```
ClassName.__mro__
```

79. What is the diamond problem?

The diamond problem happens when a class receives the same base class through multiple paths.

```
  A
 / \
B   C
 \ /
  D
```

Python solves it using MRO.

80. What is a mixin class?

A mixin is a small class that adds one specific feature to another class.

```
class LoggingMixin:
    def log(self, message):
        print(message)
```

A mixin is normally not used as a complete standalone object.

F. Polymorphism

81. What is polymorphism?

Polymorphism means one method or operation can have different behaviours for different objects.

```
developer.work()
tester.work()
```

Both use work(), but their work is different.

82. What is method overriding?

Method overriding happens when a child class provides its own version of a parent method.

```
class Employee:
    def work(self):
        print("Employee works")


class Developer(Employee):
    def work(self):
        print("Developer writes code")
```

83. What is method overloading?

Method overloading means using the same method name with different parameter lists.

Python does not support traditional signature-based overloading directly.

84. How can we achieve overloading-like behaviour?

We can use:

- Default arguments
- *args
- **kwargs
- Type-based dispatch

```
def add(self, *numbers):
    return sum(numbers)
```

85. Overloading vs overriding?

Overloading	Overriding
Same method, different parameters	Child rewrites parent method
Usually inside one class	Requires inheritance

Overloading	Overriding
Not directly supported in Python	Supported in Python

86. What is duck typing?

Duck typing means Python cares about an object's behaviour, not its exact class.

If an object has the required method, Python can use it.

87. Give a duck-typing example.

```
class Employee:
    def work(self):
        print("Employee works")


class Robot:
    def work(self):
        print("Robot works")


def start_work(worker):
    worker.work()
```

Both `Employee` and `Robot` can be passed to `start_work()`.

88. What is operator overloading?

Operator overloading defines how operators work with custom objects.

```
def __add__(self, other):
    return self.salary + other.salary
```

Now two employee objects can use `+`.

89. Can we create a new operator in Python?

No. We can change the behaviour of existing operators for custom objects, but we cannot create a completely new operator.

90. What is runtime polymorphism?

Runtime polymorphism means Python decides which method to call while the program is running.

Method overriding and duck typing are common examples.

91. Does Python have compile-time polymorphism?

Python does not provide traditional compile-time method overloading because it is dynamically typed.

92. What should an overloaded operator return for an unsupported type?

It should usually return:

```
NotImplemented
```

This allows Python to try another valid operation or raise a proper error.

G. Abstraction

93. What is abstraction?

Abstraction hides complicated internal details and shows only necessary features.

Example: A user calls `withdraw()` without knowing the bank's internal processing.

94. How is abstraction implemented in Python?

It is commonly implemented using:

- `ABC`
- `@abstractmethod`

from the `abc` module.

95. What is an abstract class?

An abstract class is a parent class that defines common rules for child classes.

It may contain both:

- Abstract methods

- Normal implemented methods

96. What is an abstract method?

An abstract method is a method that child classes must implement.

```
from abc import ABC, abstractmethod


class Employee(ABC):
    @abstractmethod
    def calculate_salary(self):
        pass
```

97. Can we create an object of an abstract class?

We cannot create its object when it contains unimplemented abstract methods.

98. Can an abstract class have a constructor?

Yes. An abstract class can have `__init__()` and normal instance variables.

99. Can an abstract class have normal methods?

Yes. It can contain both abstract and concrete methods.

100. What is a concrete class?

A concrete class is a complete class whose object can be created.

It must implement all required abstract methods.

101. What happens if a child does not implement an abstract method?

The child also remains abstract, and its object cannot be created.

102. What is an interface?

An interface defines methods that implementing classes are expected to provide.

Python does not have a separate interface keyword. We commonly use:

- Abstract base classes
- Protocols
- Duck typing

103. Abstraction vs encapsulation?

Abstraction	Encapsulation
Hides complicated logic	Protects internal data
Focuses on what an object does	Focuses on how data is controlled
Uses abstract classes	Uses private members and properties

H. Object Relationships

104. What is association?

Association means two independent objects work together.

Example: An Employee works on a Project.

Both objects can exist separately.

105. What is aggregation?

Aggregation is a weak has-a relationship.

Example: A Department has Employees, but employees can exist without that department.

106. What is composition?

Composition is a strong has-a relationship.

Example: A Car has an Engine that it strongly owns.

107. Aggregation vs composition?

Aggregation	Composition
Weak ownership	Strong ownership
Child can exist independently	Child is strongly connected to owner
Department has Employees	Car has Engine

108. Inheritance vs composition?

Inheritance	Composition
Is-a relationship	Has-a relationship
Developer is an Employee	Car has an Engine
Reuses through parent class	Reuses by containing objects

109. Which is better: inheritance or composition?

It depends on the relationship.

- Use inheritance for a true **is-a** relationship.
- Use composition for a **has-a** relationship.

Composition is often more flexible.

I. Magic Methods

110. What are magic methods?

Magic methods are special methods with double underscores.

Examples:

```
__init__
__str__
__len__
__add__
```

They are also called dunder methods.

111. What is __str__()?

__str__() returns user-friendly text for an object.

```
def __str__(self):
    return f"Employee: {self.name}"
```

It is used by print() and str().

112. What is __repr__()?

__repr__() returns a detailed representation mainly for developers and debugging.

```
def __repr__(self):
    return f"Employee(name='{self.name}')"
```

113. __str__() vs __repr__()?

__str__()	__repr__()
For users	For developers
Easy-to-read output	Detailed debugging output
Called by str()	Called by repr()

114. What is __len__()?

It controls the result of len(object).

```
def __len__(self):
    return len(self.employees)
```

115. What is __call__()?

__call__() allows an object to behave like a function.

```
class Calculator:
    def __call__(self, a, b):
        return a + b


calculator = Calculator()
print(calculator(10, 20))
```

116. What is __eq__()?

__eq__() controls the == operator.

```
def __eq__(self, other):
    return self.employee_id == other.employee_id
```

117. What is __lt__()?

__lt__() controls the < operator.

```
def __lt__(self, other):
    return self.salary < other.salary
```

118. What is __getitem__()?

It allows an object to use indexing.

```
class Team:
    def __init__(self, members):
        self.members = members

    def __getitem__(self, index):
        return self.members[index]
```

Now:

```
team[0]
```

works.

J. Advanced OOP Questions

119. What is a dataclass?

A dataclass is a class mainly used to store data.

It can automatically create:

- __init__()
- __repr__()
- __eq__()

```
from dataclasses import dataclass


@dataclass
class Employee:
    name: str
    salary: float
```

120. What is an immutable object?

An immutable object cannot be changed after creation.

Examples:

- Integer
- String
- Tuple
- Frozen dataclass

121. How can we create an immutable dataclass?

```
@dataclass(frozen=True)
class Employee:
    employee_id: int
    name: str
```

frozen=True prevents normal attribute modification.

122. What is shallow copy?

A shallow copy creates a new outer object but shares nested objects.

```
copy.copy(object)
```

123. What is deep copy?

A deep copy creates independent copies of the outer and nested objects.

```
copy.deepcopy(object)
```

124. Shallow copy vs deep copy?

Shallow copy	Deep copy
Nested objects are shared	Nested objects are copied
Uses `copy.copy()`	Uses `copy.deepcopy()`
Faster and lighter	Uses more time and memory

125. What is `is` vs `==`?

- `is` checks whether two variables point to the same object.
- `==` checks whether their values are equal.

```
a = [1, 2]
b = [1, 2]

print(a == b)  # True
print(a is b)  # False
```

126. What is object introspection?

Introspection means checking information about an object while the program runs.

Useful functions:

```
type()
id()
dir()
getattr()
hasattr()
isinstance()
```

127. What does getattr() do?

It gets an attribute using its name as a string.

```
name = getattr(employee, "name")
```

128. What does setattr() do?

It sets an attribute using its name as a string.

```
setattr(employee, "salary", 50000)
```

129. What does hasattr() do?

It checks whether an object has an attribute.

```
print(hasattr(employee, "salary"))
```

130. What does delattr() do?

It removes an attribute from an object.

```
delattr(employee, "temporary_value")
```

131. What is __dict__?

__dict__ stores many object or class attributes in dictionary form.

```
print(employee.__dict__)
```

Possible output:

```
{"name": "Aamir", "salary": 50000}
```

132. What is __slots__?

__slots__ limits normal instance attributes and can reduce memory usage.

```
class Employee:
    __slots__ = ("name", "salary")

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

It is useful when creating many similar objects.

133. What is a metaclass?

A metaclass is a class used to create classes.

By default, Python classes are created by type.

```
print(type(Employee))
```

Metaclasses are advanced and are mainly used in frameworks and libraries.

134. What is coupling?

Coupling means how strongly classes depend on each other.

- High coupling: classes are strongly dependent.
- Low coupling: classes can change more independently.

Low coupling is usually easier to maintain.

135. What is cohesion?

Cohesion means how closely a class's responsibilities are related.

High cohesion means a class focuses on one clear job.

Example: SalaryCalculator should calculate salary, not send emails and manage databases too.

K. SOLID Principles

136. What is SOLID?

SOLID contains five design principles for writing maintainable OOP code.

137. What is the Single Responsibility Principle?

A class should have one main responsibility.

Example:

- EmployeeRepository saves employee data.
- SalaryCalculator calculates salary.

One class should not do every job.

138. What is the Open/Closed Principle?

A class should be open for extension but closed for unnecessary modification.

Easy meaning:

> Add new behaviour without breaking existing code.

139. What is the Liskov Substitution Principle?

A child object should safely replace its parent object.

If Developer is an Employee, it should behave correctly wherever an Employee is expected.

140. What is the Interface Segregation Principle?

Do not force a class to implement methods it does not need.

Use small, focused interfaces instead of one very large interface.

141. What is the Dependency Inversion Principle?

High-level classes should depend on general rules or abstractions, not directly on one fixed low-level class.

Easy meaning:

> Depend on common behaviour, not one specific implementation.

L. Common Scenario-Based Questions

142. When should you use a class?

Use a class when:

- Data and actions belong together.
- Many similar objects are needed.
- You need inheritance or encapsulation.
- You are representing a real entity.

143. When should you use inheritance?

Use inheritance when there is a true is-a relationship.

Example:

```
Developer is an Employee
```

Do not use inheritance only to reuse a small amount of code.

144. When should you use composition?

Use composition when one object has or uses another object.

Example:

```
Car has an Engine
```

145. Why should salary be private?

Salary is sensitive data. Making it private allows controlled access and validation through methods or properties.

146. Why is an abstract class useful?

It ensures that related child classes follow the same rules.

For example, every employee type must implement calculate_salary().

147. How would you design different employee types?

Create an abstract Employee parent and child classes such as:

- FullTimeEmployee
- PartTimeEmployee
- Freelancer

Each child implements its own salary calculation.

148. How would you prevent negative salary?

Use a property setter.

```
@salary.setter
def salary(self, amount):
    if amount < 0:
        raise ValueError("Salary cannot be negative")
    self.__salary = amount
```

149. How would you count created objects?

Use a class variable.

```
class Employee:
    count = 0

    def __init__(self):
        Employee.count += 1
```

150. How would you compare two employee objects?

Implement __eq__().

```
def __eq__(self, other):
    return self.employee_id == other.employee_id
```

M. Important Output-Based Questions

151. What is the output?

```
class Employee:
    company = "ABC"


e1 = Employee()
e2 = Employee()

e1.company = "XYZ"

print(e1.company)
print(e2.company)
```

Output:

```
XYZ
ABC
```

e1.company becomes an instance attribute.

152. What is the output?

```
class Employee:
    skills =


e1 = Employee()
e2 = Employee()

e1.skills.append("Python")

print(e2.skills)
```

Output:

```
['Python']
```

The class-level list is shared.

153. What is the output?

```
class Parent:
    def show(self):
        print("Parent")


class Child(Parent):
    def show(self):
        print("Child")


Child().show()
```

Output:

```
Child
```

The child overrides the parent method.

154. What is the output?

```
class A:
    def show(self):
        print("A")


class B(A):
    def show(self):
        print("B")


class C(A):
    def show(self):
        print("C")


class D(B, C):
    pass


D().show()
```

Output:

```
B
```

MRO starts with:

```
D → B → C → A
```

155. What is the output?

```
class Employee:
    def __init__(self, name):
        self.name = name


e1 = Employee("Aamir")
e2 = Employee("Aamir")

print(e1 == e2)
```

Output:

```
False
```

The objects are different. Define `__eq__()` to compare their data.

156. What is the output?

```
class A:
    value = 10


a = A()
b = A()

a.value = 20
A.value = 30

print(a.value, b.value)
```

Output:

```
20 30
```

`a` has its own instance value. `b` reads the class value.

---

157. What is the output?

```
class A:
    def show(self):
        print("A")


class B(A):
    def show(self):
        print("B")
        super().show()


B().show()
```

Output:

```
B
A
```

The child method runs first and then calls the parent method.

N. Top 20 Answers to Memorize

1. **OOP:** Programming using classes and objects.
2. **Class:** Blueprint used to create objects.
3. **Object:** Instance of a class.
4. **Attribute:** Variable belonging to an object or class.
5. **Method:** Function defined inside a class.
6. **`self`:** Current object.
7. **`cls`:** Current class.
8. **`__init__()`:** Initializes an object.
9. **Encapsulation:** Protects and controls data.
10. **Inheritance:** Child reuses parent features.
11. **Polymorphism:** Same method, different behaviour.
12. **Abstraction:** Hides unnecessary details.
13. **Overriding:** Child rewrites a parent method.

14. **Overloading:** Same method name with different parameters; not directly supported in Python.
15. **Duck typing:** Required behaviour matters more than exact class.
16. **`super()`:** Calls the next method according to MRO.
17. **MRO:** Order used to search for methods.
18. **Aggregation:** Weak has-a relationship.
19. **Composition:** Strong has-a relationship.
20. **Dataclass:** Class that automatically generates common data methods.